Machine Learning and Embedding Geometry

Lecture 2

Friday March 6, 2026

NEXT: **Lecture 3**, Tuesday, March 10: 16:30–18:00 CET
Zoom Link: https://unipd.zoom.us/j/85246961830
Meeting ID: **852 4696 1830**

Supervised Learning as Function Optimization

To learn, we must make choices …

What is the Task we are trying to do? What is the <u>mathematical representation</u> of the Task? What data is relevant?

Consider a mapping between two domains:

$$X \xrightarrow{f} Y \qquad f : X \to Y$$

$$f : x \mapsto y, \text{ where } x \in X, \; y \in Y.$$

Represent the Task as a *Risk or Loss function* that defines the impact of being wrong: $R\left(\hat{f}(x), y\right)$ *Alignment problem*

The mapping f is unknown, however N associated pairs between the spaces X and Y are available.

Training data: $\mathcal{D} = \left\{(x_1, y_1), \ldots, (x_N, y_N)\right\}$ *Data sufficiency, Sampling complexity*

Select a parametric representation for the unknown function, f: $f_\theta(x)$ *Expressivity*

"Learn" by minimizing the Loss function, R, over the training data: $\min_\theta \mathrm{E}_{\mathcal{D}}\left[R\left(f_\theta(x), y\right)\right]$ *Fitting algorithm*

Since we optimized w.r.t the Loss function, we need to test with hold-out data: $\mathrm{E}_{\sim\mathcal{D}}\left[R\left(f_{\theta^*}(x), y\right)\right]$ *Post hoc uncertainty estimate*

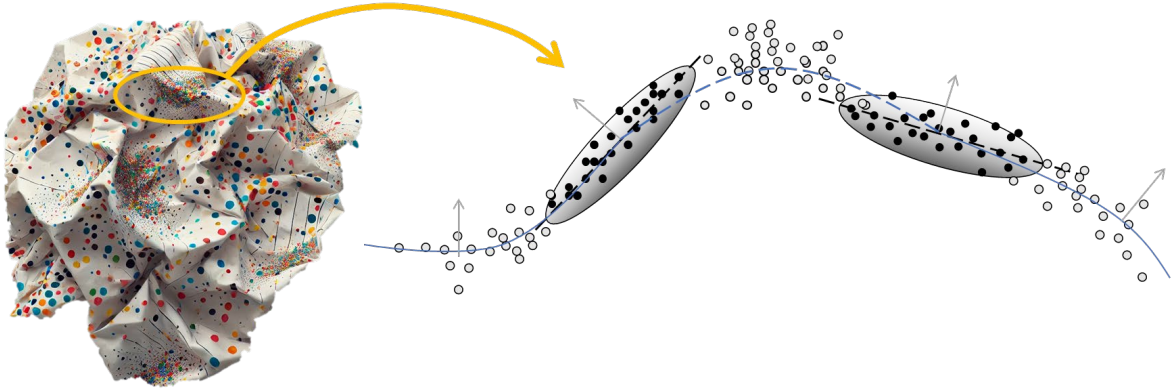
Probability approximated as Geometry

Create/Learn an Embedding or Feature Vector space that encodes information about *statistical similarity as distances*.

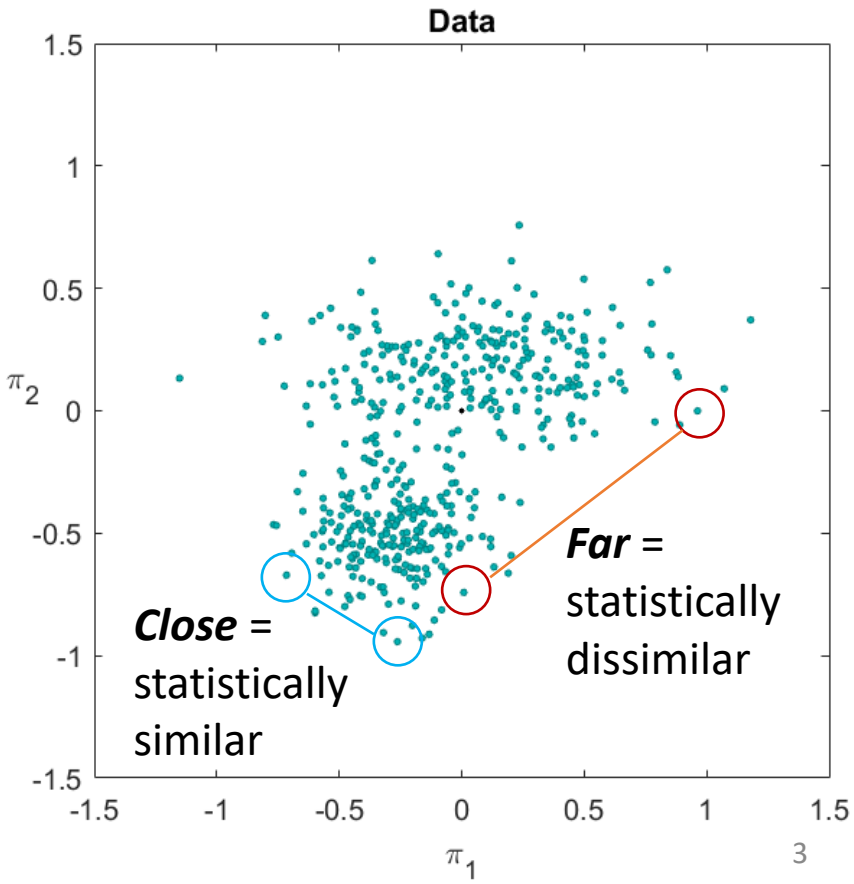
$$-\log p_{\text{similarity}}\left(y, y'\right) \approx \frac{1}{2}\left\|\mathbf{y}-\mathbf{y}'\right\|_2^2$$

$$E : \mathcal{D} \to \mathbb{R}^D$$

$$E : y \mapsto \mathbf{y}$$



Data sufficiency is a local question about having enough samples to track the local curvature induced by the embedding.



Classical ML: Geometry First, Partition Second

Framework: $$f(x) = g(E(x)) = g \circ E(x)$$

- E: fixed embedding.
- g: learned partition or function.

Examples:
- PCA + regression.
- Kernel methods.
- Fixed Fourier/wavelet features.
- Local polynomial regression.

Properties:
- Geometry is explicit.
- Partition complexity is measurable.
- Attribution is clear.
- Bias from embedding can be separated from variance in partition.

Philosophically: We choose the coordinate system. Then we learn inside it.

Modern Deep Learning: Learn the Geometry

Deep networks remove the clean separation

$$f(x) = g_\phi\left(E_\theta(x)\right) = g_\phi \circ E_\theta(x)$$

Both E and g are learned jointly adjusting their respective parameters, ϕ and θ, to minimize the loss function, R.

$$\min_\theta \mathrm{E}_{(x,y)\sim\mathcal{D}}\left[R\left(f_\theta(x), y\right)\right]$$

Only the loss function, L, is fixed.
This is a conceptual shift:
We no longer choose the geometry.
We let the loss shape it.

Consequences

- Geometry is task-shaped.
- Distances are not intrinsic.
- Embedding reflects loss, not ontology.
- Attribution becomes entangled.

New option: *Transfer Learning*
Assume that representations, E_θ, learned for one task can apply to a different task.

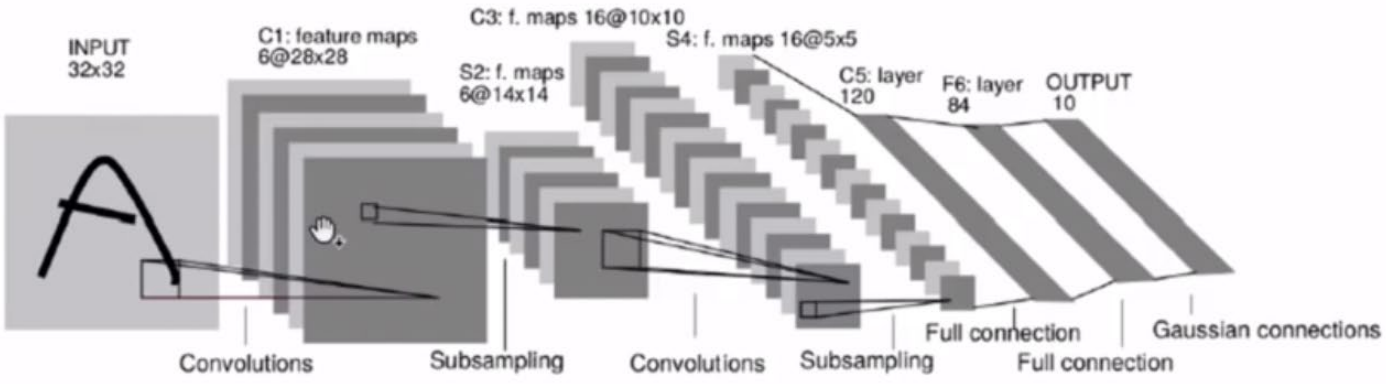
Surrogate objective function, R → Train to optimize R → Reuse transformation, E, of data to features for data not originally used to co-train g and E but learn a new h:

$$y = h_\phi \circ E_\theta(x)$$

Convolutional Neural Networks

Key properties of CNNs

Images
Patches
Pixels



- ☺ Convolutional **(Translation invariance)**
- ☺ Scale Separation **(Compositionality)**
- ☺ Filters localized in space **(Deformation Stability)**
- ☺ $\mathcal{O}(1)$ parameters per filter (independent of input image size n)
- ☺ $\mathcal{O}(n)$ complexity per layer (filtering done in the spatial domain)
- ☺ $\mathcal{O}(\log n)$ layers in classification tasks

Transformers: Dynamic Geometry

Transformers go further …

They begin with baseline embeddings:
- Token embeddings.
- Positional encodings.

Then repeatedly update representations via the *Attention Mechanism*:
Each token representation becomes a weighted combination of others.

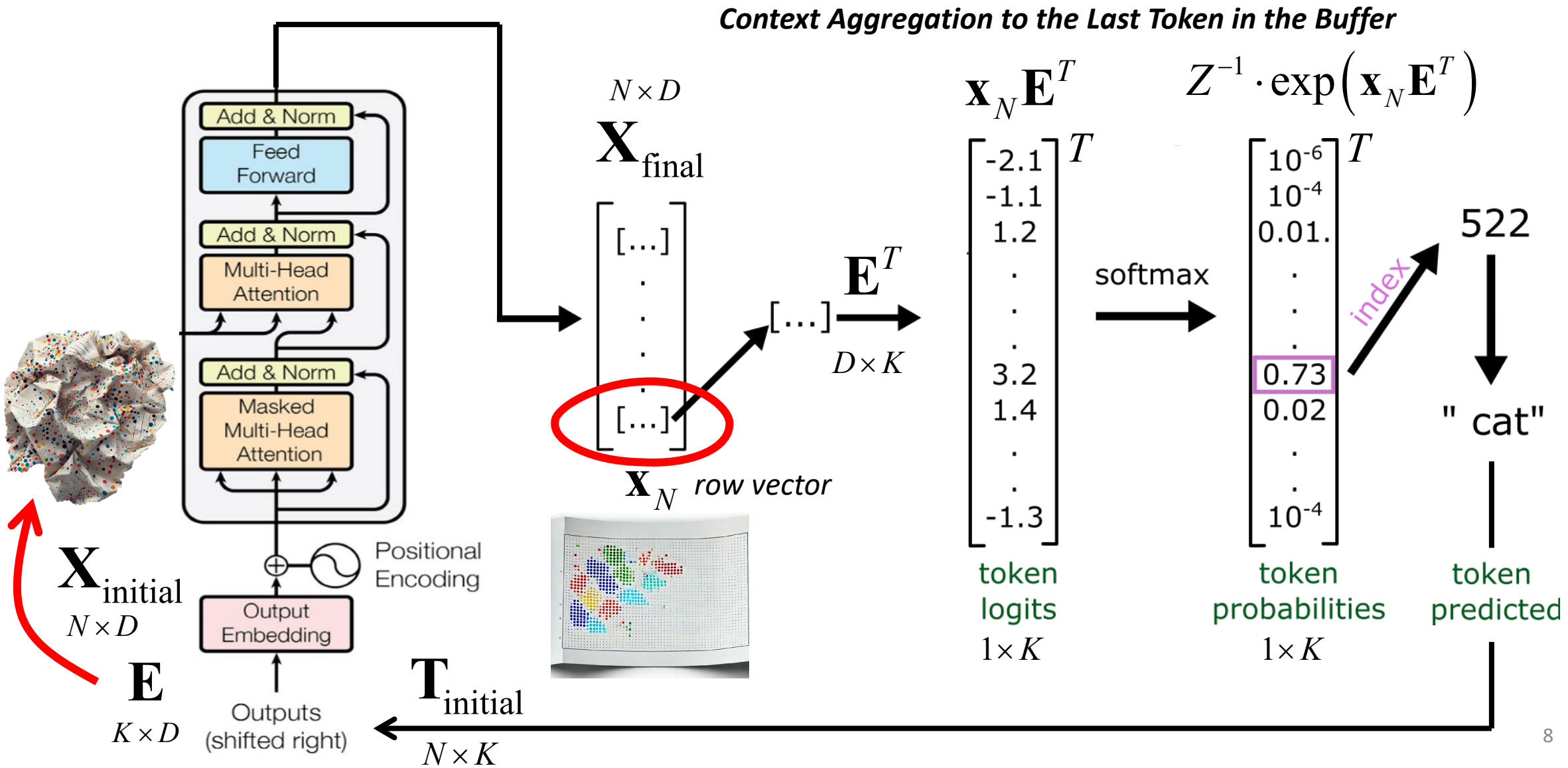
Meaning:
- Geometry is recomputed per input.
- Neighborhoods are dynamic.
- Distance is contextual.

Progression:
1. Classical: fixed geometry.
2. Deep nets: learned geometry.
3. Transformers: dynamically reconstructed geometry.

Partition at the top is often linear.
The heavy lifting is geometric warping.

Key insight:
The embedding is provisional.
Context determines geometry.
Powerful — but interpretability and stability suffer.

Transformer's "Algorithmic Skepticism" of the Embedding



Diffusion Models: Learn the Geometry of Density

Diffusion takes a different path …

Instead of partitioning space:

1. Add noise until distribution becomes Gaussian.
2. Learn how to reverse that process.
3. Generate data via learned reverse flow.

They learn:

$$\mathbf{F}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

The score function or a “statistical” force that drives particles from a normal distribution to that of the generative model.
This is local geometric information about density.

Contrast:

- Transformers: geometry serves prediction.
- Diffusion: geometry represents probability mass.

Diffusion models are:

- Locally estimated.
- Globally generative via integration.
- They model distribution geometry, not partitions.

Learning by diffusing: The information potential

Physics problem:

N particles are diffusing within an unknown potential energy function, $u(\boldsymbol{x})$.

→ Estimate $u(\boldsymbol{x})$ from a single observation of the N particle positions

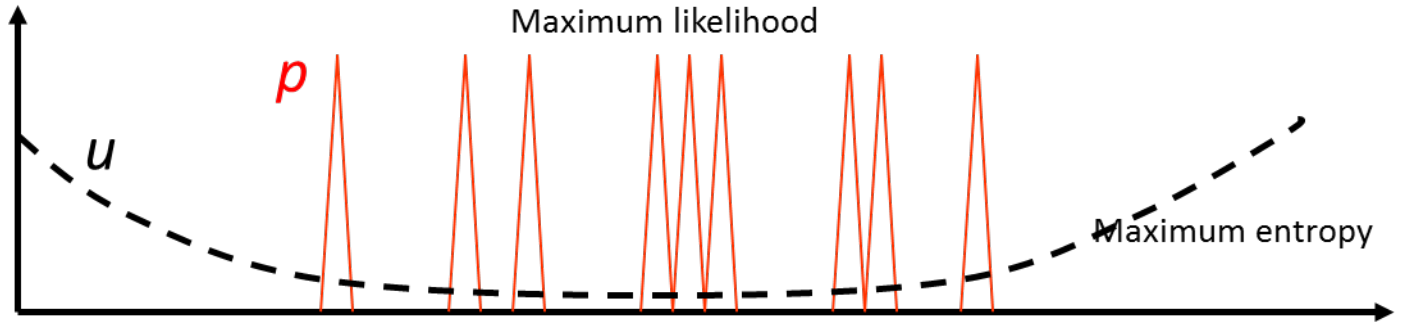
Drift-diffusion Eqn.:

$$\partial_t p = \nabla \cdot \left(D \nabla p + p \nabla u \right)$$

Empirical distribution as IC:

$$p(\mathbf{x},0) = \frac{1}{N} \sum_{n=1}^{N} \delta(\mathbf{x} - \mathbf{x}_n)$$

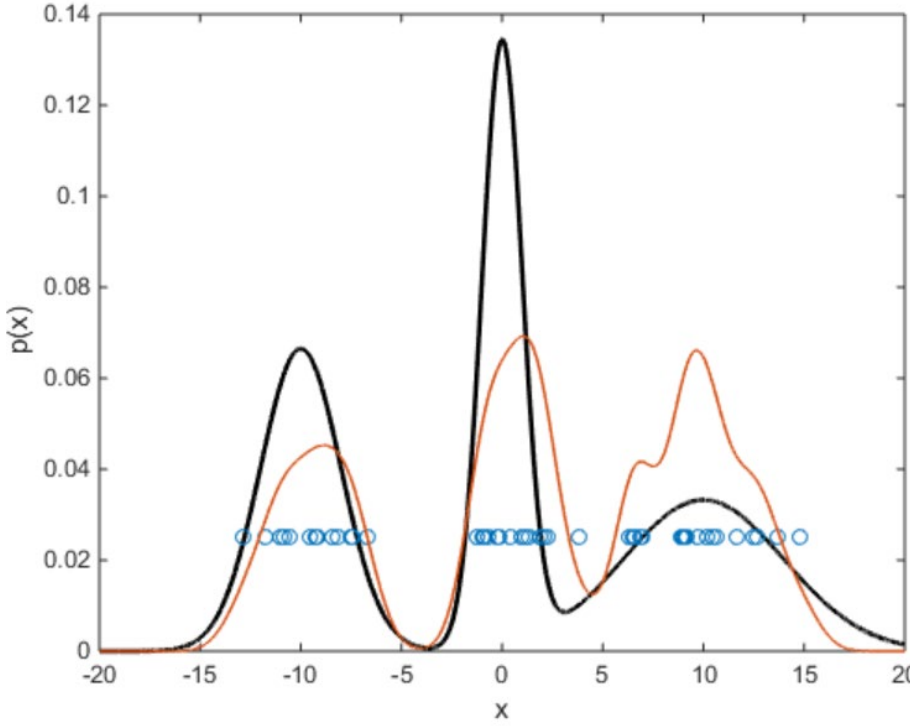
$$\lim_{t \to \infty} p(\mathbf{x},t) = \frac{e^{-u(\mathbf{x})/D}}{Z} \quad \Rightarrow \quad u(\mathbf{x}) = -\ln p(\mathbf{x}) + \ln Z$$



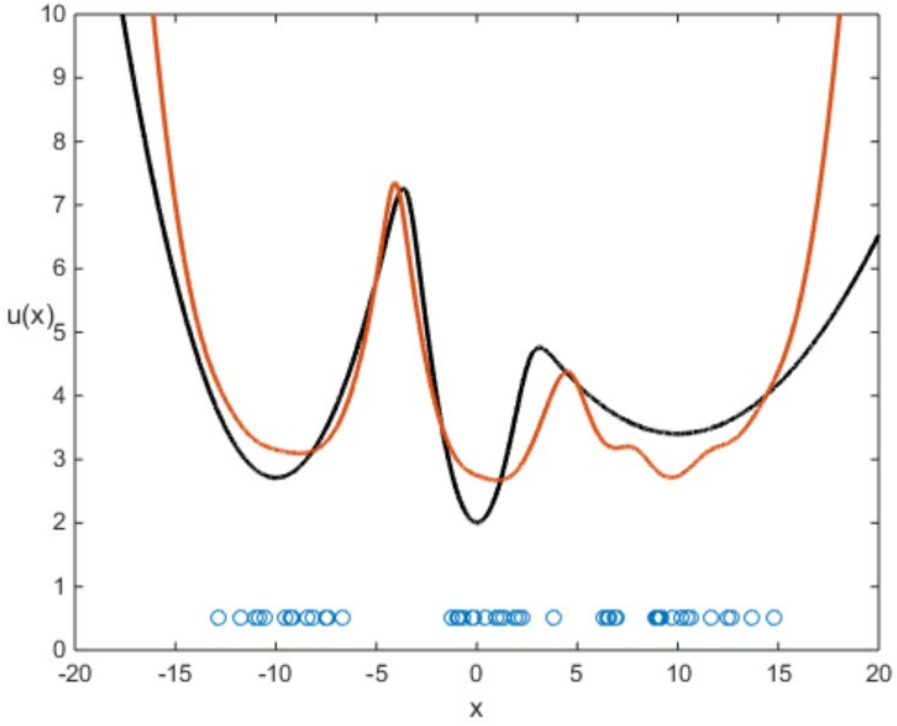
Learning by diffusing: The information potential

$$\hat{p}(\vec{x}\,|\,\vec{t}_N^{\,*}) = \frac{1}{N}\sum_{n=1}^{N}\frac{1}{\left(4\pi D t_n^*\right)^{d/2}} e^{-\frac{|\vec{x}-\vec{x}_n|^2}{4Dt_n^*}} \quad \Rightarrow \quad \mathbf{F}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

PROBABILITY DISTRIBUTION: *p*(*x*)



POTENTIAL FUNCTION: *u*(*x*)



$$u(x) = -D \ln \hat{p}(x\,|\,t_N^*)$$

The Evolution of Geometric Strategy

Summary

1. **Partition fixed geometry.**

2. **Learn geometry to simplify partition.**

3. **Dynamically rewrite geometry per input.**

4. **Abandon partition — learn stochastic flow of density.**

Epistemic Tradeoffs

More flexible geometry means:
- Greater expressive efficiency.
- Harder attribution.
- Entangled uncertainty.
- Weaker identifiability.
- Loss becomes sole anchor.

Local classical methods:
- Preserve interpretability.
- Scale with sample density.
- Indicate where more data is needed.

Modern architectures:
- Collapse representation and decision.
- Obscure geometry.
- Implicitly assume data-rich regimes.